

Herramientas de Desarrollo Profesional - TIC

Semana 2

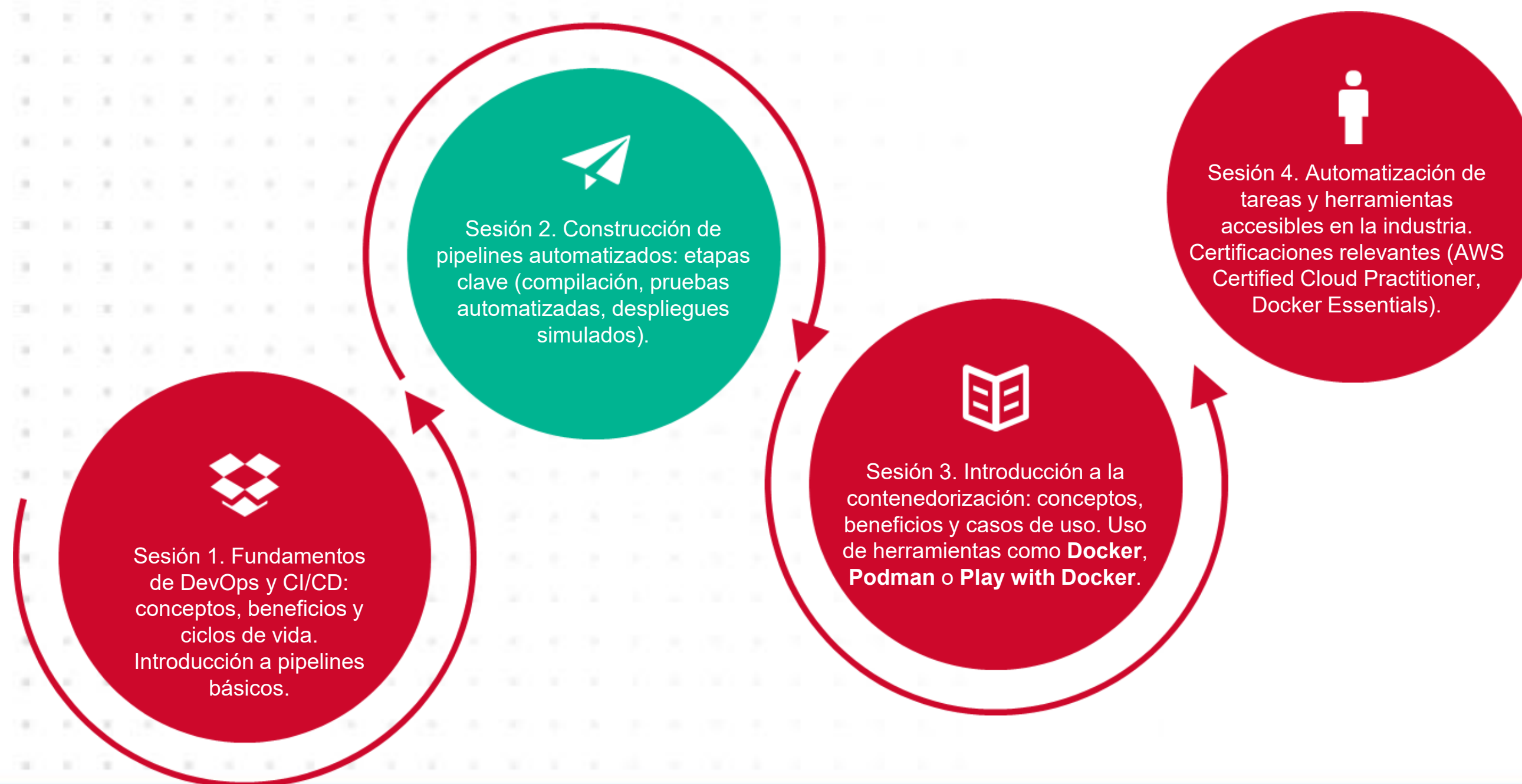
Unidad 1. DevOps y Automatización de Procesos

Profesor: Roosevelt López



Universidad
Tecnológica
del Perú

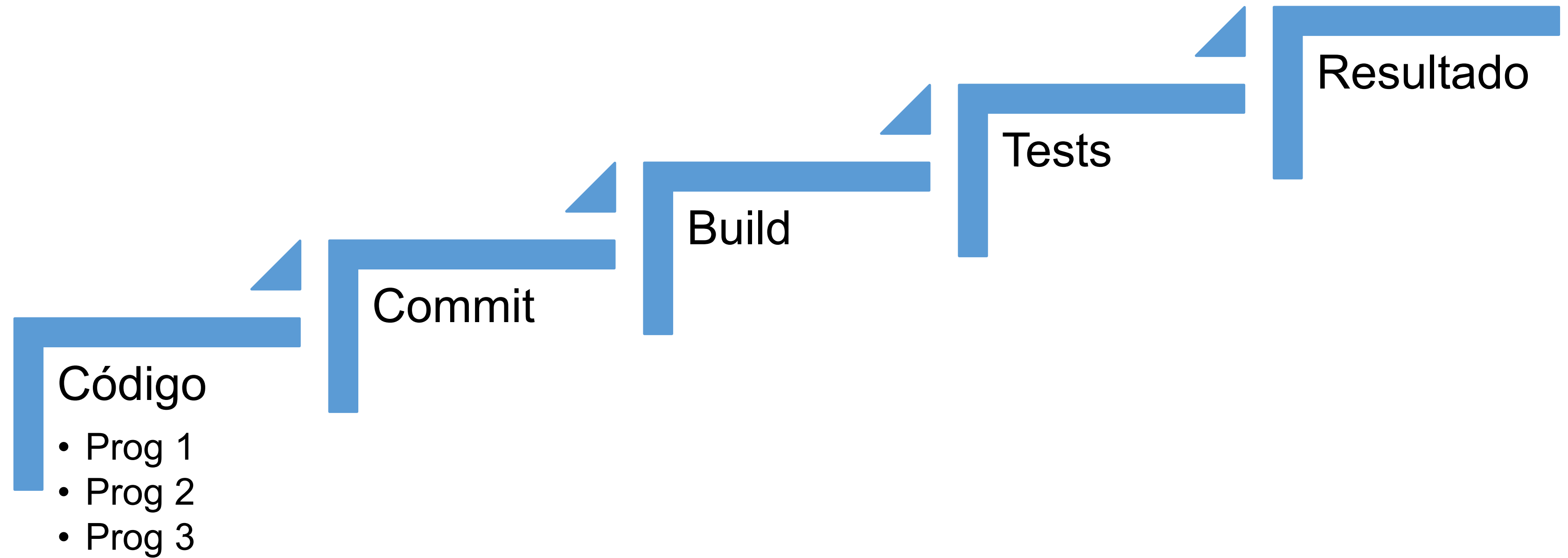
Ruta de la unidad 1: DevOps y Automatización de Procesos



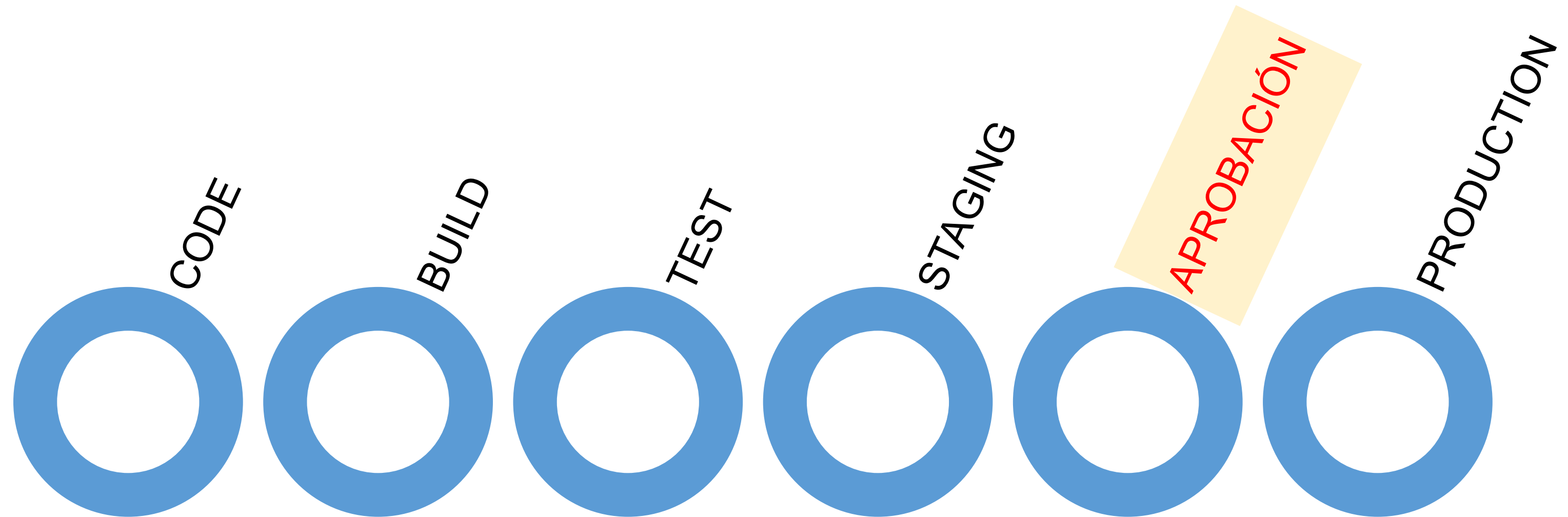
¿Tienen alguna consulta o duda sobre la clase previa?



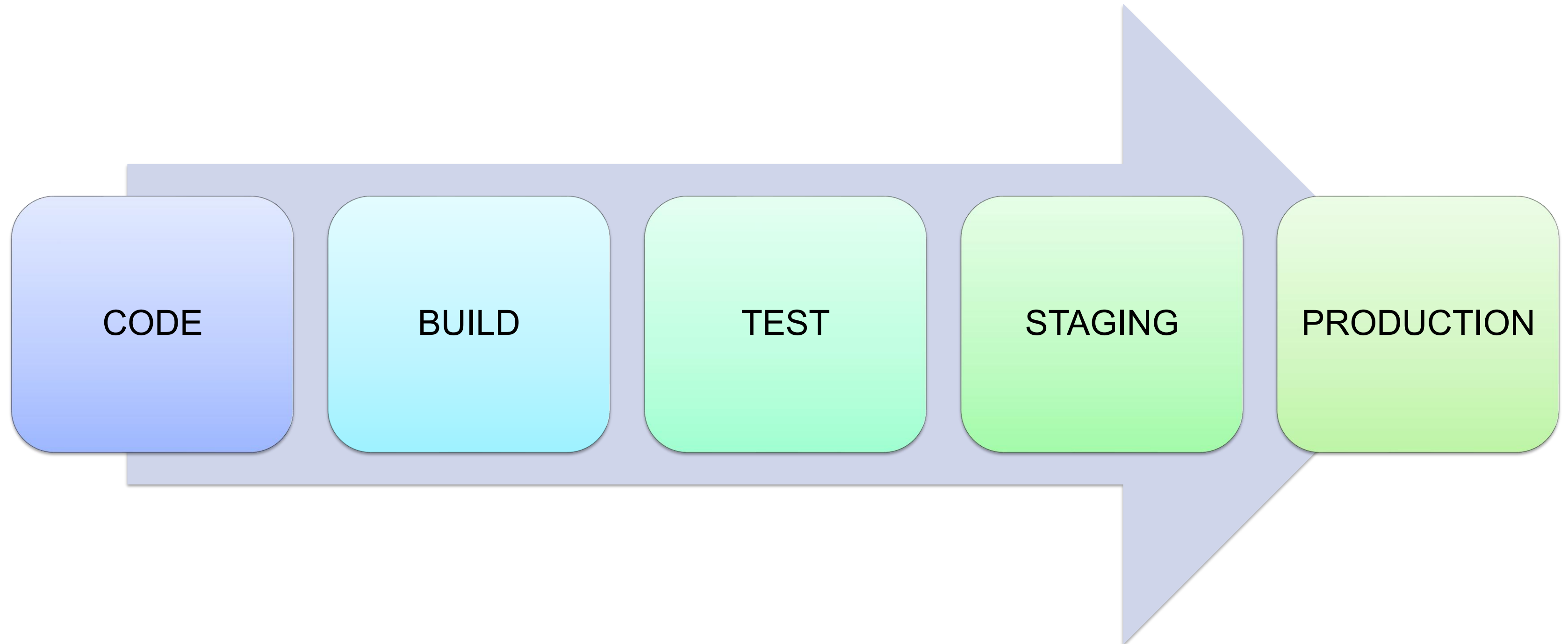
CI: Integrar constantemente



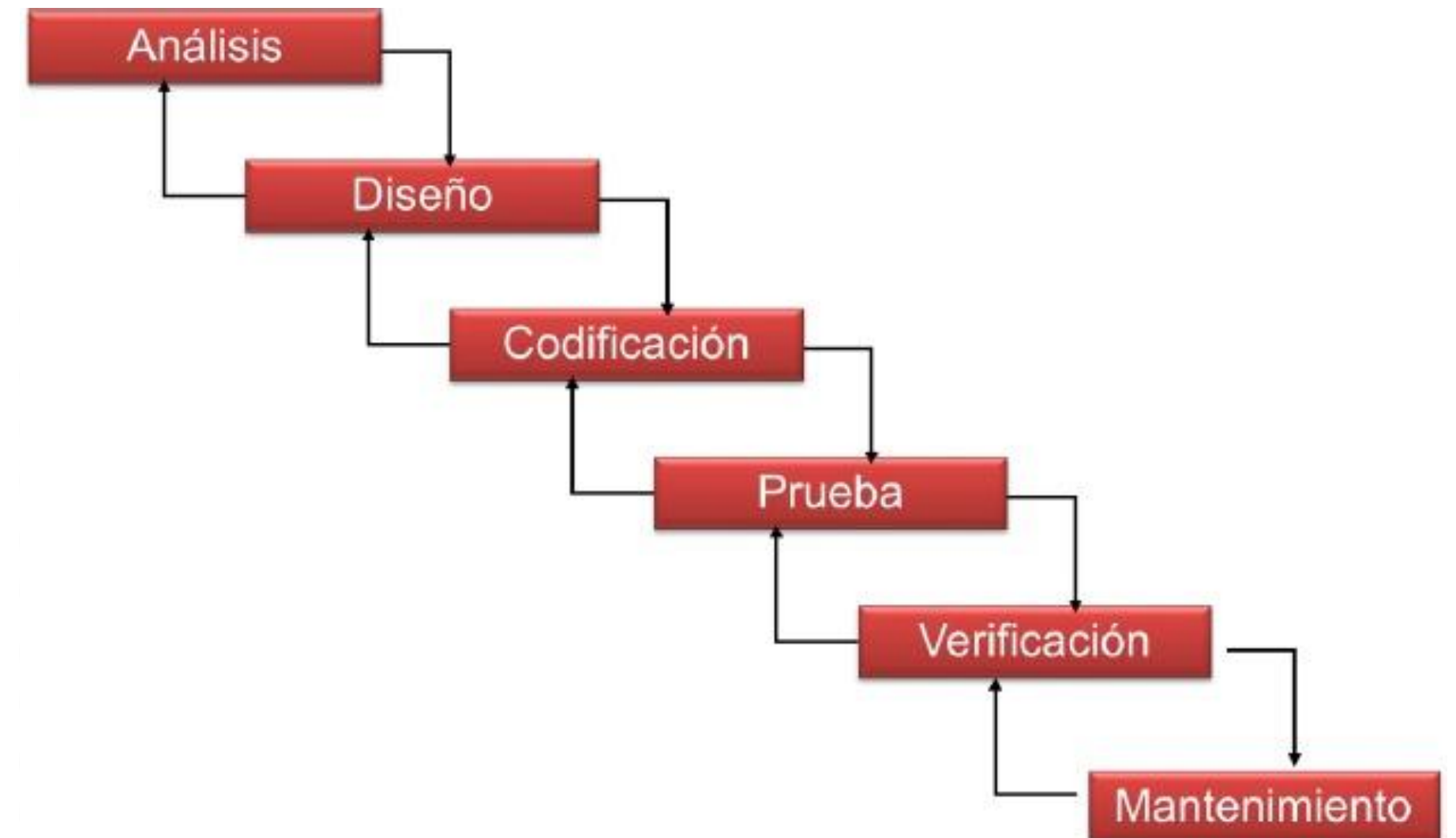
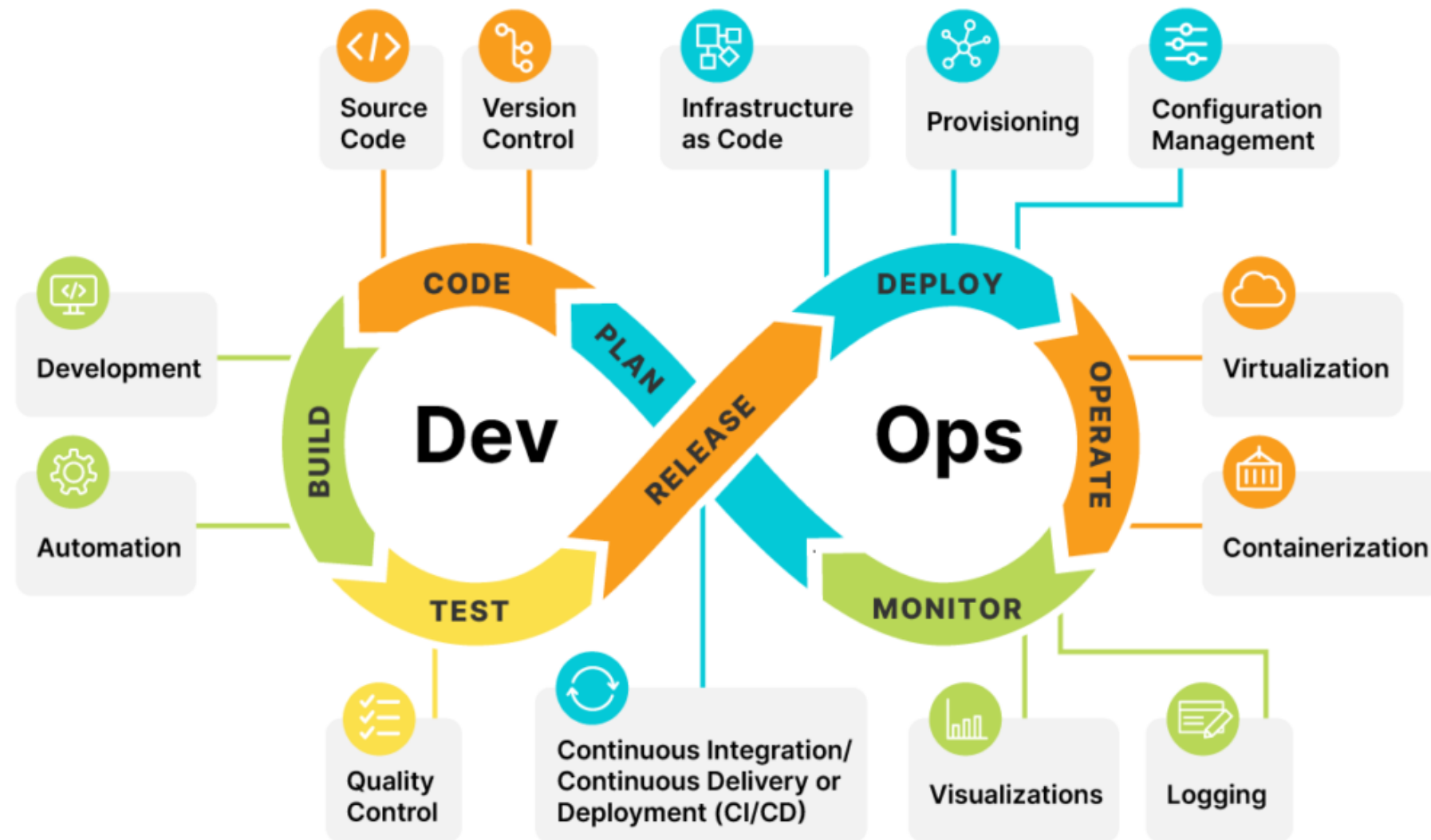
Continuous Delivery



Continuous Deployment



Comparación DevOps con el modelo tradicional (cascada)

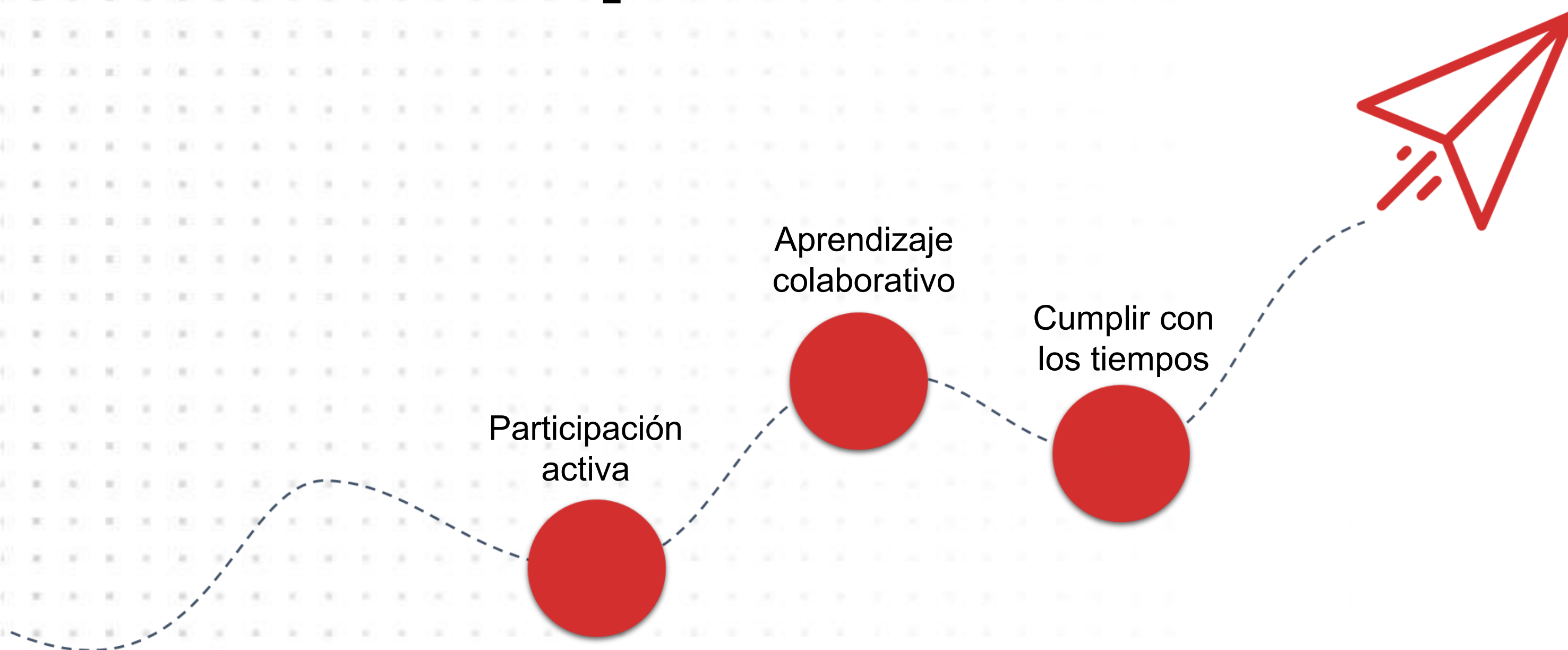


Ciclo de vida de DevOps

El ciclo de vida de DevOps se representa como un bucle continuo, donde cada fase se retroalimenta:

- **Pruebas:** Automatización de pruebas de calidad.
- **Entrega (Delivery):** Preparación para desplegar en producción.
- **Despliegue (Deployment):** Implementación automatizada.
- **Operación:** Gestión del entorno en ejecución.
- **Monitoreo:** Seguimiento del rendimiento y feedback para mejora continua.

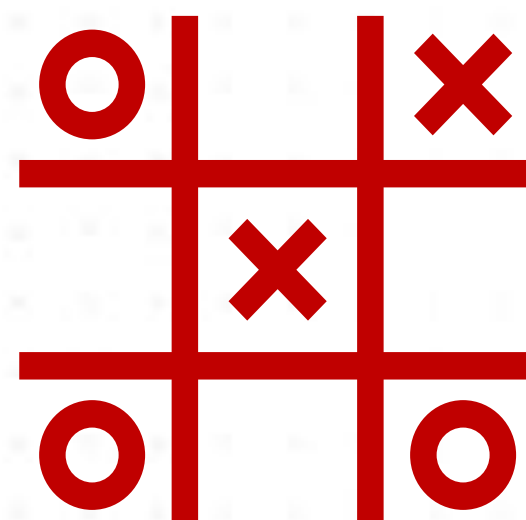
Acuerdos para el desarrollo de la sesión



Dinámica

"A gogo diga usted"

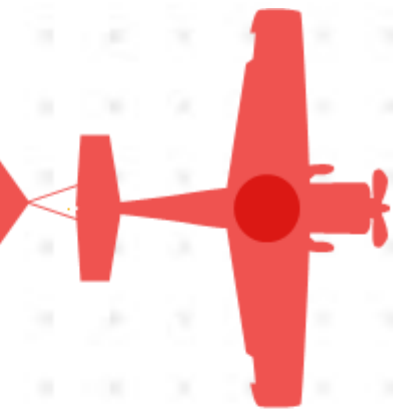
- Un programa o sitio web que automatiza procesos. *Por ejemplo, el correo electrónico que permite tener respuestas automáticas o programar envíos.*



Y ahora,

- **¿Qué pasaría si esos programas no tuvieran funciones automatizadas?**

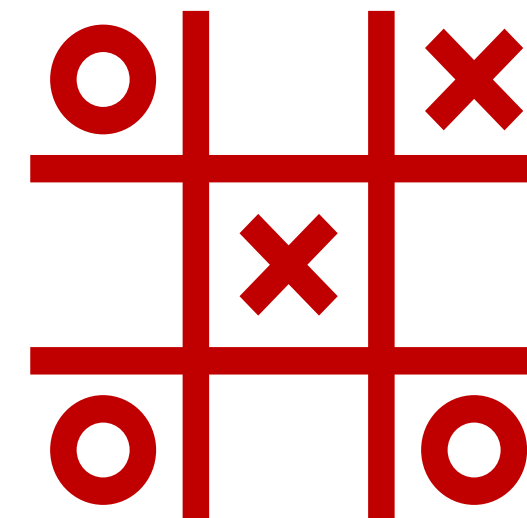
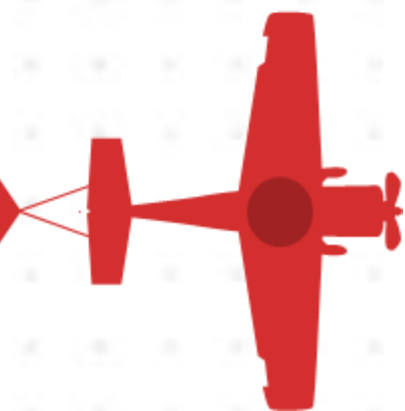
Participa



Piensa



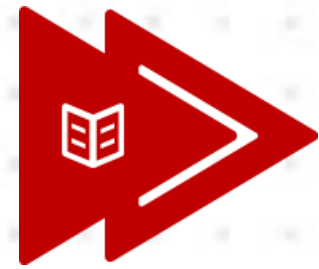
Comparte en pares



Logro de la sesión

Al finalizar la sesión, el estudiante construye un pipeline automatizado con una herramienta especializada en la resolución de un caso planteado.





Construcción de pipelines automatizados, considerando etapas clave:

- Compilación
- Pruebas automatizadas
- Despliegues simulados

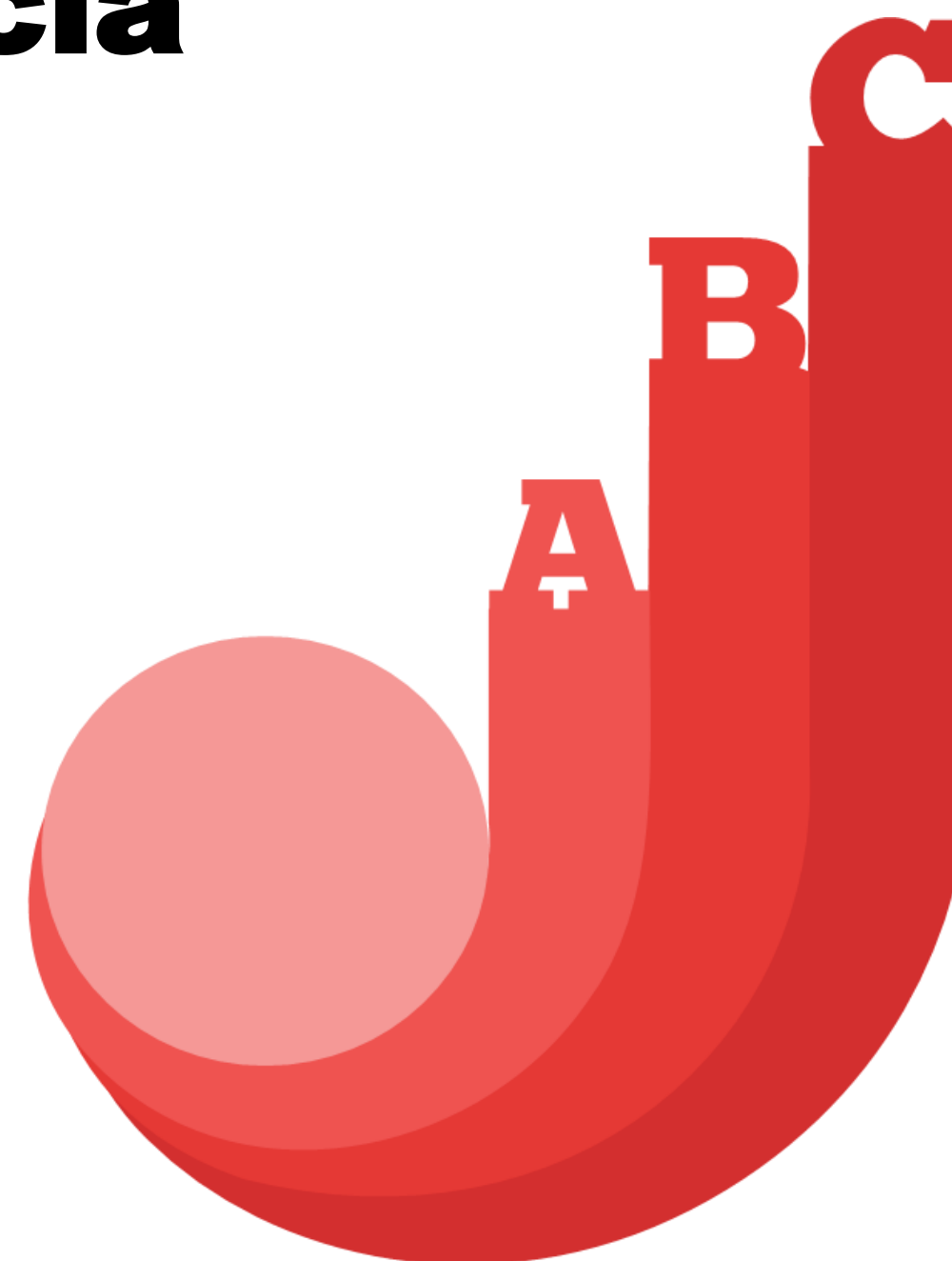


Cuéntanos una breve experiencia

A. ¿Quién ha compilado código manualmente?

B. ¿Quién ha hecho pruebas automatizadas?

C. ¿Quién ha hecho despliegues?



¿Por qué serán importantes los procesos automatizados?

Lo que
ahora sé



...

UNIDAD 01

Construcción de pipelines automatizados

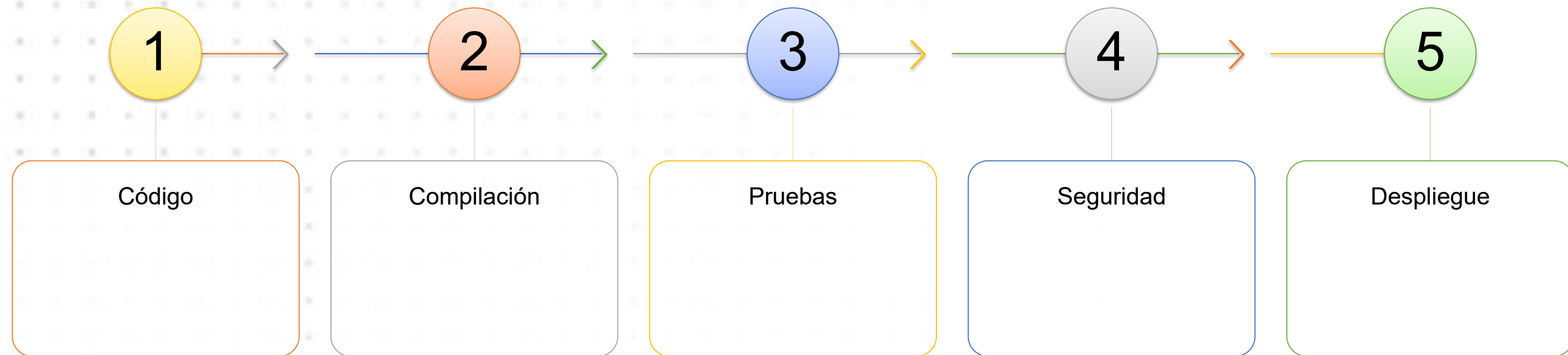
Semana 2

Pipelines y la automatización

- **Pipelines** son secuencias automatizadas de pasos que permiten compilar, probar y desplegar código de manera eficiente y coherente.
- La **automatización** en este contexto se refiere a la ejecución automática de estas tareas sin intervención manual, lo que reduce errores y acelera el proceso de desarrollo.

Etapas clave

En la construcción de Pipelines automatizados, estos se estructuran en etapas que se ejecutan en un orden específico:

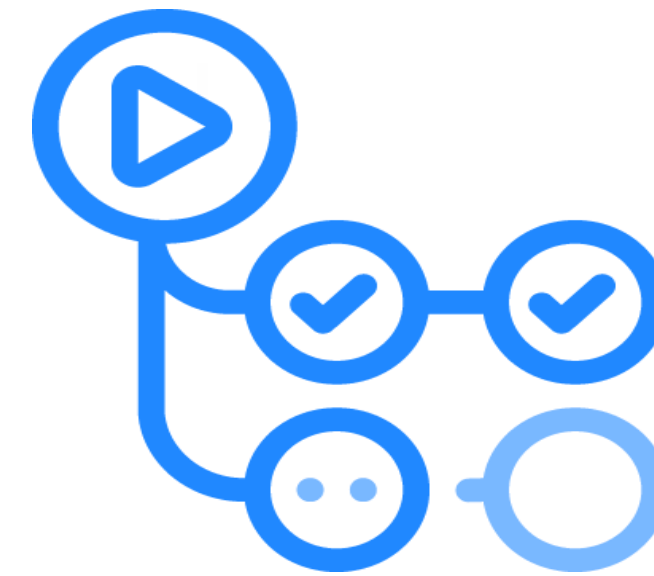


Flujo de trabajo manual versus el automatizado

Aspecto	Flujo Manual	Flujo Automatizado
Compilación	Ejecutada manualmente por desarrolladores.	Automatizada al realizar commits o pull requests.
Pruebas	Realizadas manualmente o con scripts ad hoc.	Pruebas automatizadas integradas en el pipeline.
Despliegue	Implementación manual en servidores.	Despliegue automático en entornos definidos.
Tiempo de Entrega	Lento y propenso a errores humanos.	Rápido y consistente gracias a la automatización.
Retroalimentación	Tardía, después de varias etapas.	Inmediata, con notificaciones en cada etapa del pipeline.

Herramientas

- **GitHub Actions**, permite definir workflows que se activan en respuesta a eventos en el repositorio, como pushes o pull requests, facilitando la automatización del ciclo de vida del desarrollo.
- **GitLab CI/CD**, utiliza archivos `.gitlab-ci.yml` para definir pipelines que se ejecutan en runners, permitiendo la automatización de tareas como compilación, pruebas y despliegue.



Descripción de etapas clave



Compilación

Transforma el código fuente en ejecutables o artefactos listos para pruebas o despliegue.

- **GitHub Actions:** Se utiliza el paso run dentro de un job para ejecutar comandos de compilación, como npm install o make.
- **GitLab CI/CD:** Se define un job en la etapa build que ejecuta scripts de compilación.

Descripción de etapas clave



Pruebas automatizadas

Ejecuta pruebas automatizadas para verificar la funcionalidad y estabilidad del código.

- **GitHub Actions:** Permite ejecutar pruebas utilizando frameworks como Jest o Mocha en proyectos Node.js, integrando los resultados en el workflow.
- **GitLab CI/CD:** Los jobs en la etapa test ejecutan scripts de prueba, y los resultados pueden visualizarse en la interfaz de GitLab.

Descripción de etapas clave



Despliegues simulados

Implementa los artefactos en entornos de staging o producción.

- **GitHub Actions:** Se pueden configurar jobs para desplegar en entornos de staging utilizando acciones específicas o scripts personalizados.
- **GitLab CI/CD:** La etapa deploy permite implementar los artefactos en entornos definidos, y es posible simular despliegues en entornos de prueba antes de producción.

¿Qué es GitHub Action?

Veo el vídeo

Anoto mis ideas

Comparte en pares

```
sample-app-deployment-with-github-actions on main
> git status
On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be
  (use "git checkout -- <file>..." to discard changes)

no changes added to commit (use "git add" and/or "git commit")

sample-app-deployment-with-github-actions on main
> git add .

sample-app-deployment-with-github-actions on main
> git status
On branch main
Your branch is up to date with 'origin/main'.

Changes to be committed:
  (use "git reset HEAD <file>..." to unstage)

    modified:   .github/workflows/server-deploy.yml
    modified:   index.html

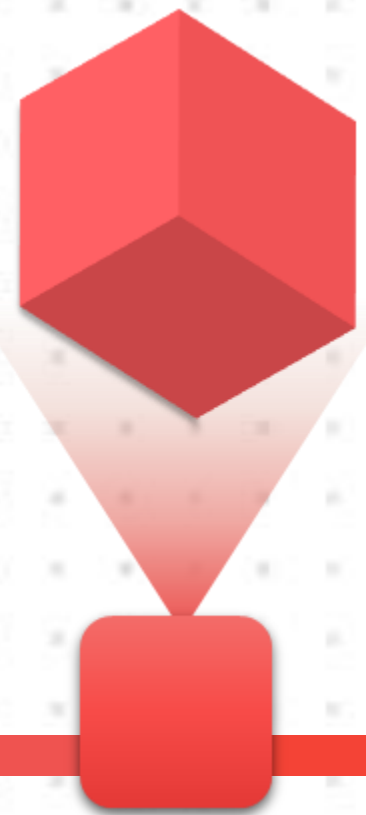
sample-app-deployment-with-github-actions on main
> git commit -m "Deploying changes!"
[main 541eb72] Deploying changes!
 2 files changed, 0 insertions(+), 1 deletion(-)

sample-app-deployment-with-github-actions on main
> git push origin main
Counting objects: 6, done.
Delta compression using up to 8 threads.
Compressing objects: 100% (4/4), done.
Writing objects: 100% (6/6), 822 bytes | 822.88 KiB
Total 6 (delta 1), reused 0 (delta 0)
remote: Resolving deltas: 100% (1/1), completed with
```


Discusión

¿Qué errores ocurren cuando todo se hace manualmente?

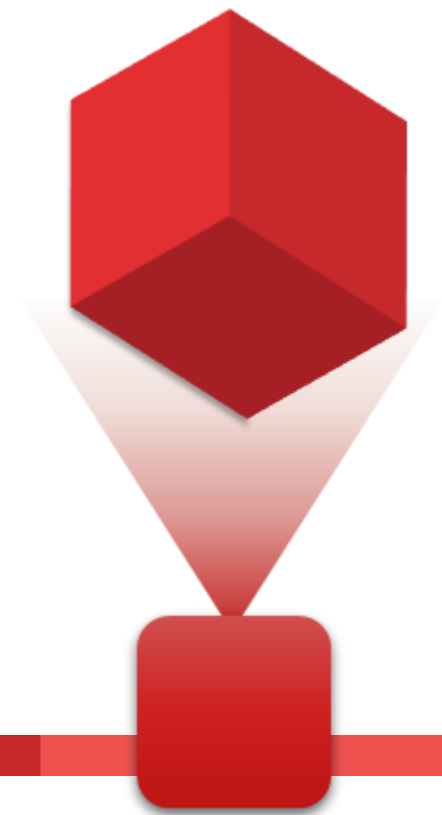
¿Qué beneficios trae automatizar estas etapas?



Pienso

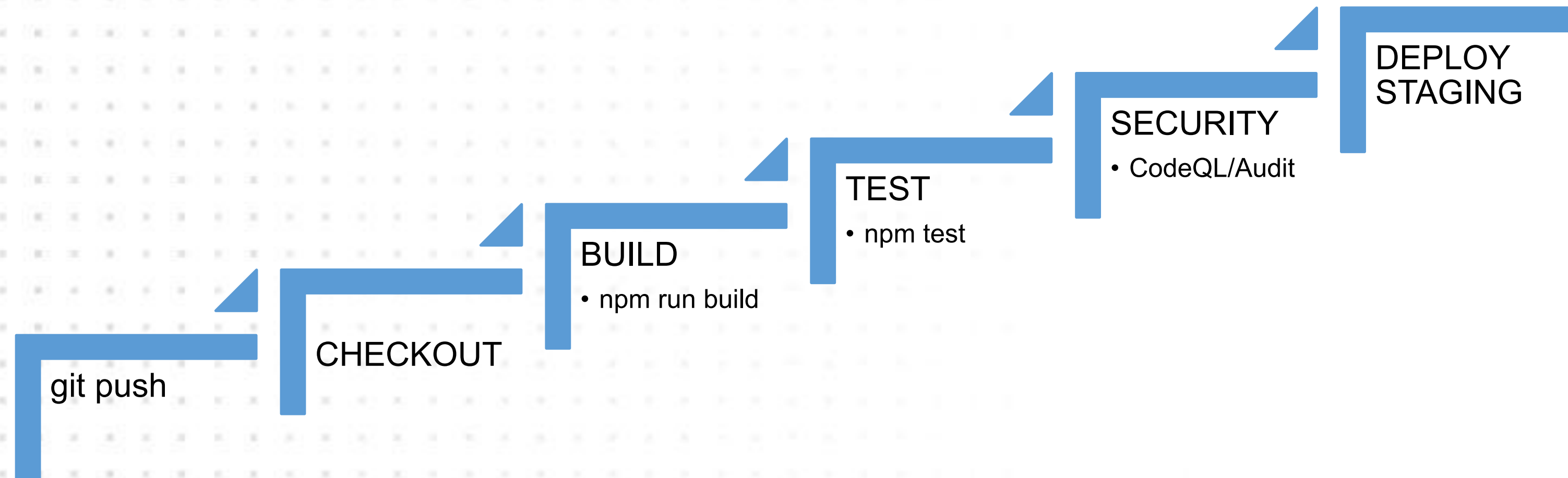


Anoto mis
ideas



Comparte
en pares

GITHUB ACTIONS



Cada etapa es una puerta

Plantilla (estructura y código base)

Código YAML para GithubAction: .github/workflows/ci.yml

```
mi-proyecto/  
├── src/  
│   └── index.js  
├── tests/  
│   └── index.test.js  
├── package.json  
├── .github/  
│   └── workflows/  
│       └── ci.yml  
└── .gitlab-ci.yml
```

```
name: CI  
  
on:  
  push:  
    branches: [ main ]  
  pull_request:  
    branches: [ main ]  
  
jobs:  
  build:  
    runs-on: ubuntu-latest  
    steps:  
      - uses: actions/checkout@v4  
      - name: Instalar dependencias  
        run: npm install  
      - name: Ejecutar pruebas  
        run: npm test
```

Experiencia Grupal

- Usando GitHub Actions o GitLab CI, los estudiantes crearán un pipeline básico que incluya:
 - Compilación de un proyecto simple (por ejemplo, en JavaScript o Python).
 - Ejecución de pruebas automatizadas (unitarias).
 - Despliegue simulado (por ejemplo, imprimir un mensaje de “Despliegue exitoso” o copiar archivos a una carpeta de staging).
- Desarrollar el caso planteado en equipos.
- **Límite de tiempo: 40 minutos**



- 1.ABRIR INTELIJ
- 2.CONECTAR GITHUB
- 3.ABRIR PROYECTO DESCUENTOS
- 4.SHARE PROJECT ON GITHUB
- 5.CREAR: descuentos-actions
- 6.SUBIR PROYECTO
- 7.CREAR: .github/workflows/pipeline.yml
- 8.COMMIT
- 9.PUSH
- 10.GITHUB ACTIONS SE EJECUTA

Comandos Útiles terminal



```
git config --global user.name "NOMBRE APELLIDO"  
git config --global user.email correo@gmail.com
```

```
git config --global user.name  
git config --global user.email
```

```
git status
```

```
git commit -m "Proyecto inicial - Sistema de descuentos"
```

```
git branch -M main  
git branch
```

```
git remote -v
```

```
git pull --rebase origin main
```

```
git add .  
git commit -m "Modificar descuento fijo a 20 soles"  
git push
```

main

1 Branch 0 Tags

Go to file

T

Add file

<> Code



Rvlation Proyecto inicial - Sistema de descuentos

bfa26c8 · 6 minutes ago

1 Commit



src/com/example/openclose

Proyecto inicial - Sistema de descuentos

6 minutes ago



.gitignore

Proyecto inicial - Sistema de descuentos

6 minutes ago

README

1. Haz clic en **Add file**.
2. Selecciona **Create new file**.
3. En el campo del nombre escribe exactamente: `github/workflows/pipeline.yml`
4. Copiar el contenido del pipeline.yml
5. Clic en Commit changes.



Universidad
Tecnológica
del Perú

w

All workflows

Showing runs from all workflows

Filter workflow runs



Help us improve GitHub Actions

Tell us how to make GitHub Actions work better for you with three quick questions.

Give feedback



1 workflow run

Workflow ▾

Event ▾

Status ▾

Branch ▾

Actor ▾



Agregar pipeline GitHub Actions

Pipeline - Proyecto Descuentos #1: Commit [ae123c1](#) pushed by [Rvlation](#)

main

📅 now

🕒 8s



🔍 Filter workflow runs

Showing runs from all workflows

 **Help us improve GitHub Actions**
Tell us how to make GitHub Actions work better for you with three quick questions.

[Give feedback](#) 

3 workflow runs		Workflow ▾	Event ▾	Status ▾	Branch ▾	Actor ▾
✓	Modificar descuento fijo a 20 soles Pipeline - Proyecto Descuentos #3: Commit 09549bf pushed by Rvlation	main		1 minute ago 11s		...
✓	Update GitHub Actions to use latest versions Pipeline - Proyecto Descuentos #2: Commit 26e5b51 pushed by Rvlation	main		5 minutes ago 13s		...
✓	Agregar pipeline GitHub Actions Pipeline - Proyecto Descuentos #1: Commit ae123c1 pushed by Rvlation	main		10 minutes ago 8s		...

pipeline

succeeded 11 minutes ago in 8s

Search logs



- > Set up job 1s
- > 1. Descargar código 1s
- > 2. Configurar Java 0s
- ▼ 3. Compilar aplicación 2s
 - 1 ▶ Run echo "=====
 - 16 =====
 - 17 BUILD - Compilando proyecto
 - 18 =====
 - 19 BUILD EXITOSO
- ▼ 4. Ejecutar prueba 0s
 - 1 ▶ Run echo "=====
 - 22 =====
 - 23 TEST - Probando aplicación
 - 24 =====
 - 25 Resultado obtenido:
 - 26 Precio final: S/170.00
 - 27 TEST EXITOSO
- ▼ 5. Control de seguridad 0s
 - 1 ▶ Run echo "=====
 - 17 =====
 - 18 SECURITY - Analizando código
 - 19 =====
 - 20 SECURITY CHECK EXITOSO
- > 6. Despliegue simulado 0s
- > Post 2. Configurar Java 0s
- > Post 1. Descargar código 1s
- > Complete job 0s

¿Por qué serán importantes los procesos automatizados?

Lo que
ahora sé



¿Qué hemos aprendido hoy?

¿Qué dificultades has tenido?

¿Para qué te ha servido lo aprendido?

¿Qué funcionó? ¿Qué errores encontraron?

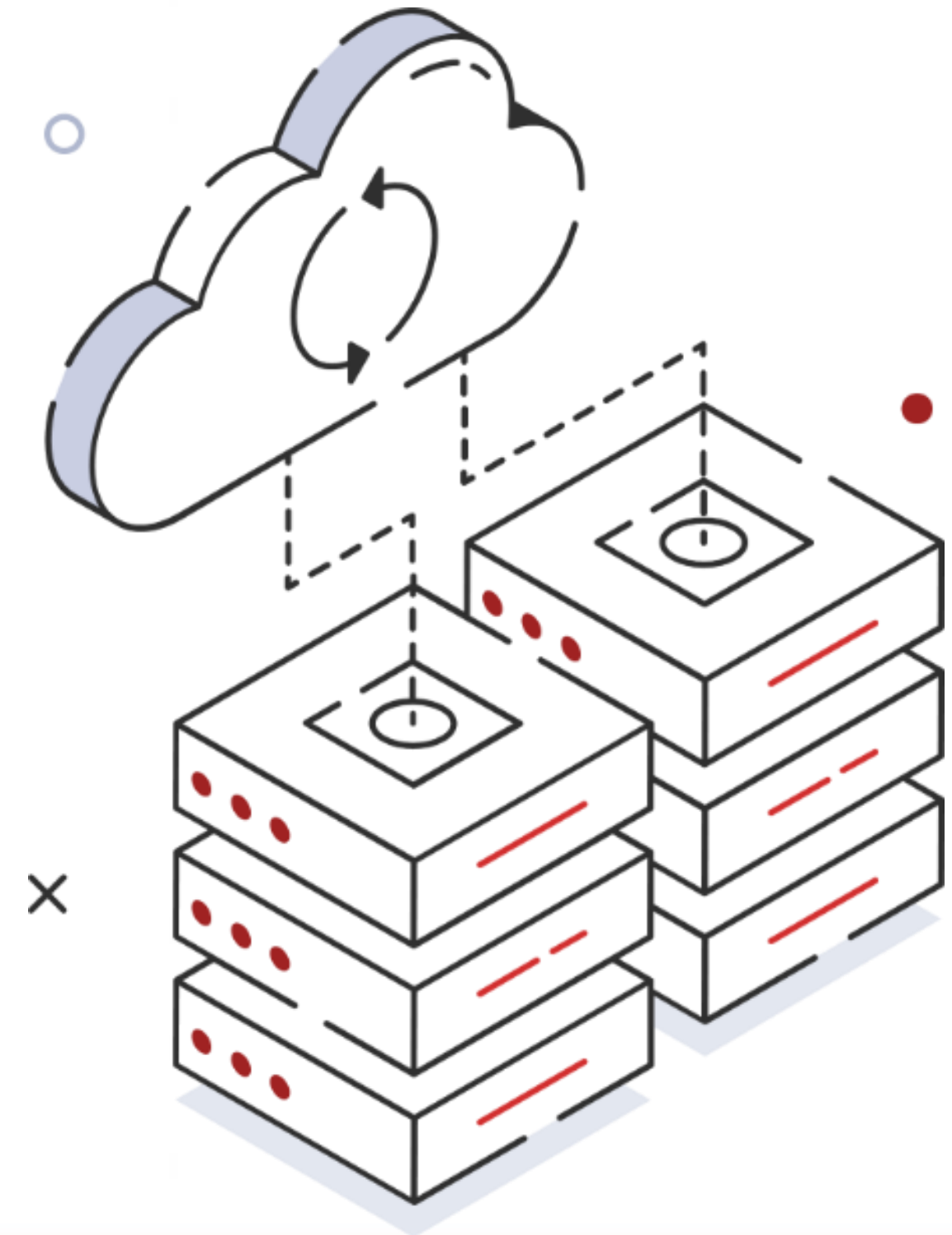
Conclusiones

- Implementar pipelines automatizados con GitHub Actions o GitLab CI/CD mejora significativamente la eficiencia y calidad en el desarrollo de software, facilitando la entrega continua y reduciendo errores humanos.



Referencias

- <https://www.ibm.com/think/topics/ci-cd-pipeline>
- <https://docs.github.com/en/actions>
- <https://docs.gitlab.com/ci/>





**Universidad
Tecnológica
del Perú**